

DAY 8 · FastAPI learning notes

Testing FastAPI Applications

TestClient · pytest · assertions · testing success and error cases · separate test files

Learning objectives

After working through these notes you should be able to:

- ✓ Explain what an automated test is and how TestClient talks to your app without a real server.
- ✓ Write pytest test functions that check status codes and JSON bodies.
- ✓ Test both successful requests and expected errors (like 422).
- ✓ Put tests in a separate `test_*.py` file and run them with `uv run pytest`.

Note

This PDF keeps the content of [FastAPI_Testing_Summary_No_Headers_Cookies.pdf](#) (which deliberately left out headers and cookies) and adds a walkthrough of `day8/main.py` and `day8/test_main.py`.

1. What is testing?

Testing means automatically sending requests to your FastAPI application and checking whether it behaves as expected. Instead of manually using a browser or Postman, you write Python test functions and let **pytest** run them.

Core flow

```
Test -> TestClient -> FastAPI application -> Endpoint
                                     |
PASS / FAIL <- Assertions <- HTTP response <-----+
```

2. TestClient

TestClient is the testing HTTP client used to make requests to your app. Create it by passing your FastAPI app to it.

Name	Mental model
<code>app</code>	the server/application being tested
<code>client</code>	the fake/test HTTP client
<code>response</code>	what the server sends back

```
from fastapi.testclient import TestClient
client = TestClient(app)
response = client.get("/items/10")
```

3. Basic tests and assertions

A test function normally starts with `test_` so pytest can discover it. `assert` checks whether a condition is true; the first false assert makes the test fail. For FastAPI the common checks are the **status code** and the **JSON body**.

```
def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"msg": "Hello World"}
```

4. Testing errors, parameters and bodies

Good tests check successful **and** failing requests. Check the status code and, when useful, the JSON error body.

```
response = client.get("/items/10")           # path parameter
response = client.get("/items/10?q=phone")   # + query parameter

response = client.get("/items/0")           # expected error
assert response.status_code == 422
```

Send a JSON request body with `json=`:

```
response = client.post("/items/", json={"id": "foobar", "title": "Foo Bar",
                                         "description": "The Foo Barbers"})
assert response.status_code == 200
```

TestClient expects data that can be converted to JSON, not a Pydantic model directly. To send model data, convert it first with `jsonable_encoder` (the same helper you used on Day 5):

```
from fastapi.encoders import jsonable_encoder
json_data = jsonable_encoder(my_model)
response = client.post("/items/", json=json_data)
```

5. Your code: the app under test

day8/main.py (imports omitted)

```
@app.get("/items/")
async def get_item():
    return {"message": "all items"}

@app.get("/items/{item_id}")
async def get_item(item_id: Annotated[int, Path(ge=1)], q: str | None = None):
    return {"item_id": item_id, "q": q}
```

- `Path(ge=1)` means `item_id` must be at least 1, so `/items/0` must produce **422** – a perfect thing to test.
- `q` has a default of `None`, so it is optional.

In your code

`TestClient`, `asyncio` and `HTTPException` are imported in `main.py` but never used there. `TestClient` belongs in the test file.

`"message"` is a typo for `"message"`. No test calls `/items/`, so it went unnoticed – a good argument for testing every route.

Both functions are named `get_item`. Each route is registered by its decorator first, so both work, but distinct names (`get_all_items`, `get_item`) are clearer.

6. Your code: the tests

day8/test_main.py (reformatted)

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_any():
    response = client.get("/items/10?q=phone")
    assert response.status_code == 200
    assert response.json() == {"item_id": 10, "q": "phone"}
    response2 = client.get("/items/10")
    assert response2.status_code == 200
    assert response2.json() == {"item_id": 10, "q": None}
    response3 = client.get("/items/0")
    assert response3.status_code == 422
```

- 1 Happy path with a query parameter: the path text **"10"** becomes the int **10**.
- 2 Optional query parameter left out: **q** is **None** (JSON **null**).
- 3 Error case: **0** breaks **Path(ge=1)** → FastAPI returns 422 on its own.

`from main import app` works because pytest adds the test file's folder to Python's import path. Run it from the project root:

```
uv run pytest day8 -v
```

Verified result: the tests pass (`test_main.py` and its commented copy `test_main_commented.py`).

Remember

Splitting `test_any` into `test_get_item`, `test_get_item_without_query` and `test_invalid_item_id` makes it obvious which case broke when a test fails.

7. Separate test files

In a real application, tests are normally placed in a separate file. A common structure is:

```
app/
  __init__.py
  main.py
  test_main.py      # from .main import app
```

Your Day 8 folder follows the same idea without a package: `main.py` + `test_main.py`, imported with `from main import app`.

8. Complete practice example (from the summary)

```

from fastapi import FastAPI, Path
from fastapi.testclient import TestClient
from typing import Annotated

app = FastAPI()

@app.get("/items/")
async def get_all_items():
    return {"message": "all items"}

@app.get("/items/{item_id}")
async def get_item(item_id: Annotated[int, Path(ge=1)], q: str | None = None):
    return {"item_id": item_id, "q": q}

client = TestClient(app)

def test_get_item():
    response = client.get("/items/10?q=phone")
    assert response.status_code == 200
    assert response.json() == {"item_id": 10, "q": "phone"}

def test_get_item_without_query():
    response = client.get("/items/10")
    assert response.status_code == 200
    assert response.json() == {"item_id": 10, "q": None}

def test_invalid_item_id():
    response = client.get("/items/0")
    assert response.status_code == 422

```

9. Common mistakes

- Forgetting to create `TestClient(app)`.
- Forgetting to assert the expected status code.
- Checking only success cases and never testing expected errors.
- Using the wrong request argument for the data you want to send (e.g. not using `json=` for a body).
- Making a required path parameter optional unnecessarily.
- Manually validating something FastAPI can validate with `Path`/Pydantic.
- Putting unrelated scenarios into one huge test; separate tests make failures easier to understand.

10. Cheat sheet & final mental model

```

client = TestClient(app)
client.get("/items/10")
client.get("/items/10?q=phone")
client.post("/items/", json={...})

assert response.status_code == 200
assert response.json() == {...}

def test_something():
    ...

```

Word	Meaning
app	FastAPI application / server

Word	Meaning
client	test HTTP client
request	what the test sends
response	what the app sends back
assert	what the test expects

The client does not decide whether your code is correct. It sends the request and gives you the response; your **assertions** decide whether the observed result matches the expected result.

11. Revision summary

- `TestClient(app)` sends fake HTTP requests straight to the app – no server needed.
- pytest runs every `test_*` function in every `test_*.py` file.
- Assert the status code and the JSON body; test error cases (e.g. 422) too.
- Send bodies with `json=`; convert Pydantic models with `jsonable_encoder` first.
- Run: `uv run pytest day8 -v`.

Source of the original summary: the FastAPI Testing tutorial (headers and cookies sections intentionally left out).